الجمهورية الجزائرية الديمقراطية الشعبية

وزارة التعليم العالي و البحث العلمي

People's Democratic Republic Of Algeria

Ministry of Higher Education and Scientific Research

MINI PROJECT REPORT

Agricultural Land Production Planning (ALPP)

Group Members

KHEFFACHE Semhane (**Leader**)

HATTABI Hadil

SOUAK Maroua

MAHFOUDIA Nour el houda Imene

May ,17nd ,2024.

Outline :

- **Introduction** 3
- **Explanation of the problem treated** 3

  Problem formulation of Part1

  Problem formulation of Part2

- **About Data** 3
- **Solution Description** 5

  Part 1 Graph Search for Best Products

  Part 2 Scenario Optimization

- **Running time** 12
- **Visualization of Graph ,Data and relations between attributes**
- **Appendix** 13

➢ Introduction:

Agricultural production planning holds an important position in food security, economic stability, and sustainable development within Algeria. This report will detail a developed strategy for the optimized agricultural use of land within the framework of assessment of agricultural strengths of each Wilaya, national consumption pattern analysis for key crops, and seasonal price variation studies. It will apply various optimization techniques in efforts to achieve different goals, such as minimizing consumer price, maximizing production, and ensuring self-sufficiency in strategic products using both uninformed and informed search strategies, and hill climbing., and through comparative analysis, the best technique will be deduced. Results from this will ensure informed decisions in the betterment of the effectiveness and sustainability of Algeria's agricultural sector.

➢ Explanation of the problem treated:

This project addresses an important and recurring challenge that can go a long way in the growth of the economy of Algeria and bring forth tremendous developments within the agricultural sector. The actual problem lies in the need to orient and rationalize agricultural production at the level of the whole country.This would be done through in-depth studies of the various important aspects, for example, knowing the agricultural strengths of each province, annual national consumption of certain commodities, and seasonal oscillations of the price of the product.The last goal is to find a balance that realizes low, affordable consumer prices, taking into consideration the market conditions, economic situation, and purchasing power of Algerians. We also look forward to increasing the produced quantities for all products to answer the need of the nation and take one further step toward food security for some strategic products in the Algerian market. From these studies and their implementation, we can systematically set up how to improve agricultural efficiency, sustainable development, and economic resilience within the agricultural sector of Algeria.

First we will tackle the searches applied on a graph to find out what a given Wilaya is best in ,so let’s consider this as the first problem and formulate it before applying different search algorithms

Problem formulation :

States: Wilayas

Initial state: given Wilaya by the user (the Wilaya we are willing to know in which product it is the best in terms of production).

Actions : compare production with another Wilaya.

Transition model: if the current Wilaya produces more, move to that Wilaya; else , stay in the current Wilaya.

Goal state: when the current Wilaya is found to be the best in producing the product among all Wilayas.

Path cost : could be the difference in production .

This is problem formulation allows for a systematic search to find the best product producing From the given Wilaya .

Second,we need to optimize and enhance farming efficiency in Algeria utilizing a meta-level tree structure. By browsing via a power structure of search specifications and also approaches, the objective is to recognize one of the most efficient combination that makes the most of farming results throughout various areas (Wilayas) and also years:

Meta-Level Tree Framework: The meta-level tree stands for a power structure of search nodes where each node represents a certain mix of search specifications or techniques. At the origin degree, the tree includes all feasible mixes of search formulas and also specifications. Each succeeding degree stands for improvements or variants of the search criteria such as various mixes of search formulas , heuristic features or expedition methods.

States: The state area includes all feasible mixes of search criteria and also approaches within the meta-level tree. Each state stands for a particular arrangement of search formulas, heuristic features, or various other search specifications.

Initial state:The preliminary state is the origin of the meta-level tree standing for the beginning factor for the search procedure. It incorporates all feasible mixes of search specifications, plus techniques.

Transition model: The activities entail browsing with the meta-level tree by choosing detailed branches and also nodes representing various mixes of search specifications along with methods.

Goal state: The transition version defines exactly how the state advances as the search formula proceeds with the meta-level tree. It includes going across from one node to one more based upon the selected activity which figures out the choice of particular search specifications plus techniques.

Path cost :The objective state is obtained when the search formula recognizes an optimum mix of search specifications and also approaches that take full advantage of the unbiased feature such as general farming efficiency and also effectiveness.

➢ About Data:

The data made use of for this job covers a duration of 4 years, from 2016 to 2019 plus makes up an extensive collection of farming information from different areas (Wilayas) in Algeria. It includes in-depth documents on a selection of farming items, catching crucial characteristics such as manufacturing quantity, land dimension, expanding period, market value, coupled with regional intake. This comprehensive information collection creates a strong structure for assessing farming fads as well as maximizing manufacturing situations to enhance the performance plus performance of Algeria's farming field. Our objective is to use this historic information to determine one of the most positive end results for every item in every Wilaya therefore educating methods for future farming preparation as well as growth.

The data source for this task was sourced from guides given with the job declaration. These publications assemble comprehensive farming documents as well as data supplying comprehensive understandings right into numerous plants as well as local farming tasks throughout Algeria. The information within these publications has actually been carefully collected together with records guaranteeing its dependability plus importance for evaluating farming fads and also maximizing manufacturing circumstances.

The data is arranged right into documents that catch necessary details regarding farming tasks throughout different areas in Algeria. Each document includes the complying with attributes:

Wilaya: The area where the information was accumulated standing for various management departments in Algeria.

Item: The kind of farming item such as wheat, barley, potatoes and so on

Production (prod): The quantity of manufacturing for the item in an offered year, gauged in suitable systems (e.g. bunches, kgs).

Dimension of Land (Superficie emblavée): The location of land made use of for growing the item normally gauged in hectares.

Season: The period throughout which the item was expanded indicating the certain duration of the year (e.g. Spring, Fall).

Price: The market value of the item reflects its worth on the market.

Consumption: The regional intake of the item showing the quantity eaten within the area.

This organized layout permits comprehensive evaluation coupled with contrast of farming methods as well as results throughout various years as well as areas.

- **Solution Description:**

The project is divided into two main parts:

Part 1:Graph Search for Best Products

The preliminary of the task concentrates on using chart search formulas consisting of Depth-First Browse (DFS), Breadth-First Browse (BFS), A * search as well as Hill Climbing, to figure out the very best item that each Wilaya creates. This includes creating a chart where each node stands for a Wilaya with its equivalent farming items based upon the information from among the 4 years (2016-2019). The individual picks the year they want to assess along with the chart. Each node consists of features such as manufacturing quantity, dimension of land made use of, market price plus regional usage for every item. The chart is after that traversed making use of the previously mentioned search formulas, examining coupled with contrasting the items created in each Wilaya to determine the very best item based upon standards such as the greatest manufacturing quantity, ideal market value. This detailed strategy enables individuals to evaluate the farming efficiency of various Wilayas for a particular year coupled with establishing one of the most effective items in each Wilaya.

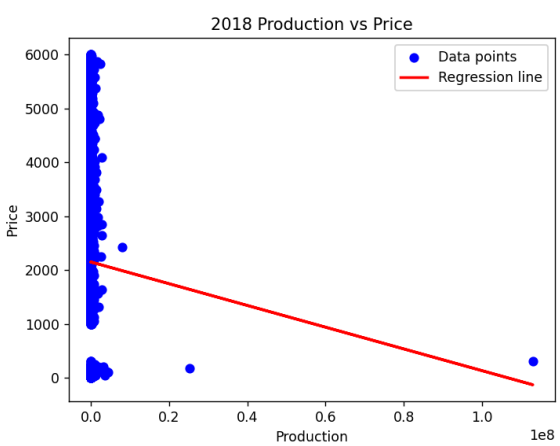
Part 2:Scenario Optimization

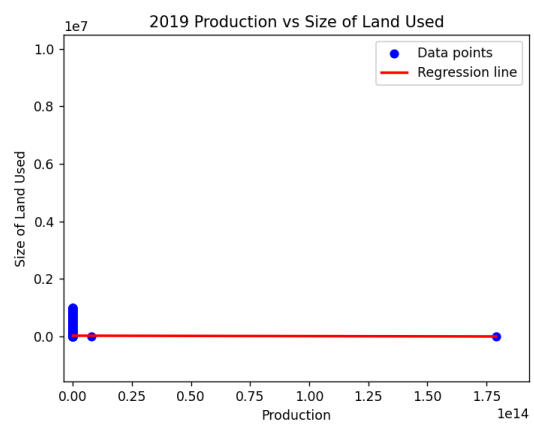
The second component of the project entails accumulating the detailed information and also using innovative search formulas to maximize the farming circumstance in Algeria. By assessing the information from all four years, the objective is to produce a circumstance that stands for the most effective feasible results for each and every item in each Wilaya. This component utilizes historic information to find out as well as recognize one of the most reliable as well as effective farming techniques, eventually developing a manufactured situation that makes the most of farming performance plus effectiveness throughout the nation.

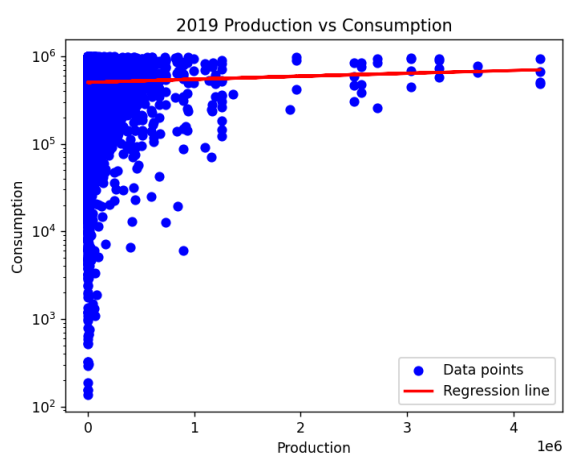
➢ Running time:

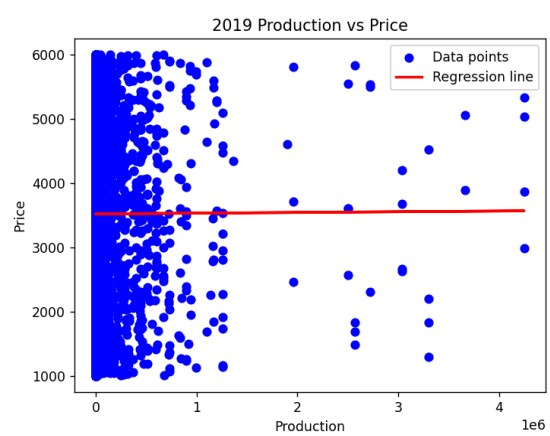
The search strategy	The time complexity
Best first search	16.20 sec
Depth first search	41.25 sec
A star A*	13.19 sec
Hill climbing	18.04 sec

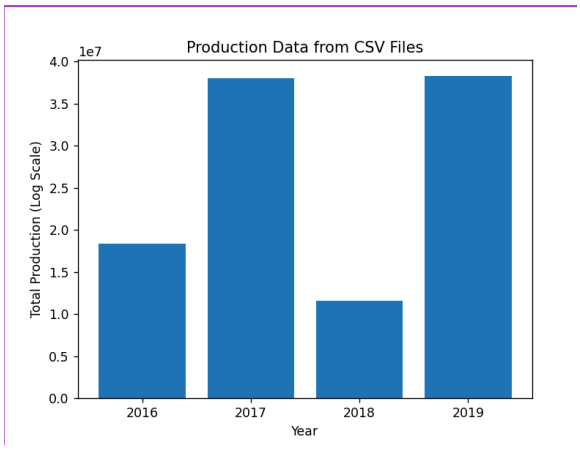
➢ Visualization of Graph ,Data and relations between attributes:










Production Data from CSV Files
1e7
Total Production (Log Scale)
Year
2016
2017
2018
2019

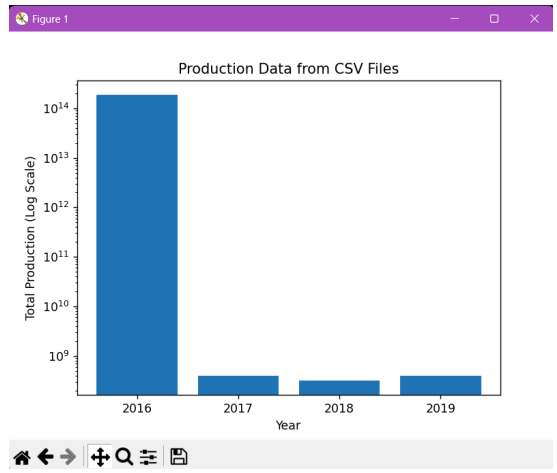
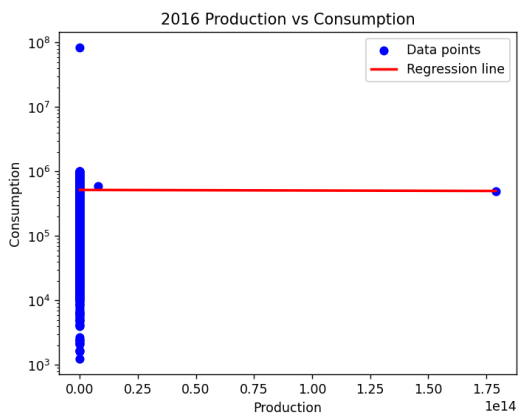
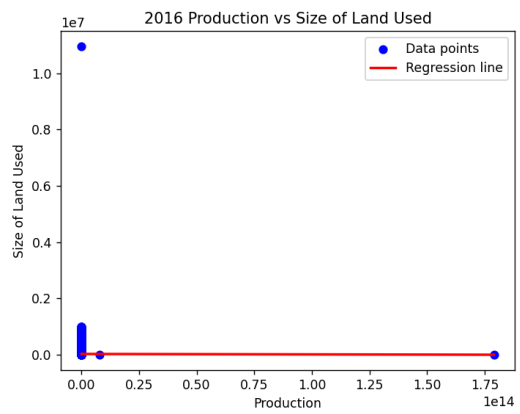
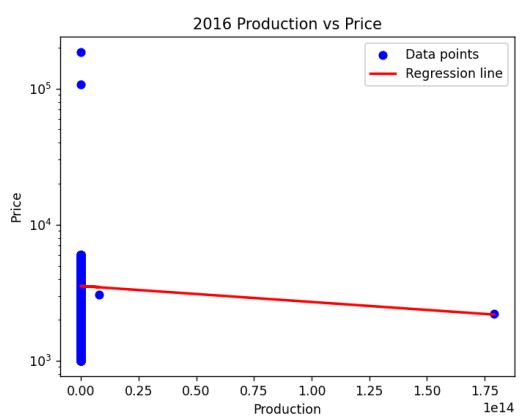
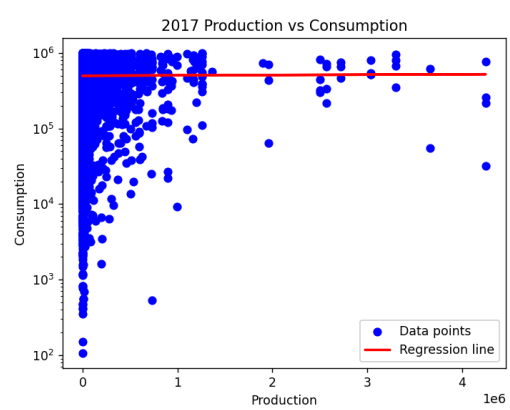


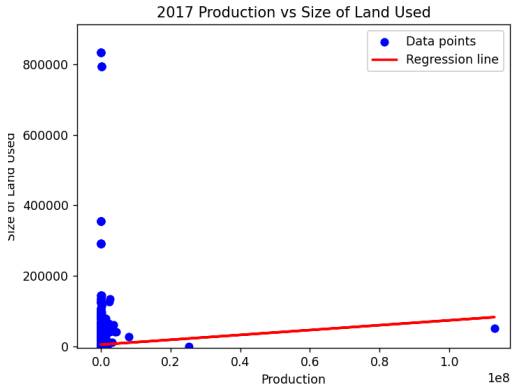
Figure 1
Production Data from CSV Files
Total Production (Log Scale)
Year
2016
2017
2018
2019

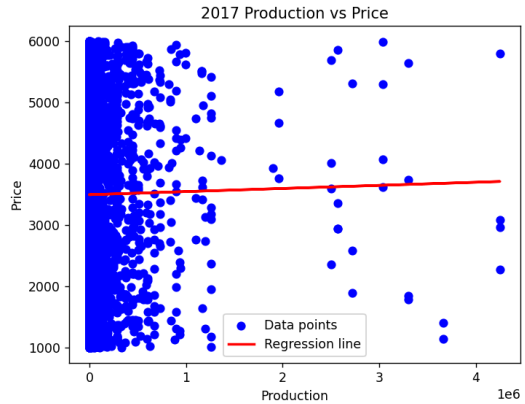

2016 Production vs Consumption
Data points
Regression line
Consumption
Production
1e14

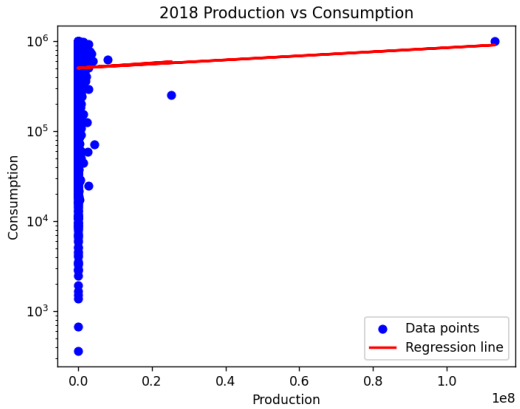

2016 Production vs Size of Land Used
1e7
Data points
Regression line
Size of Land Used
Production
1e14

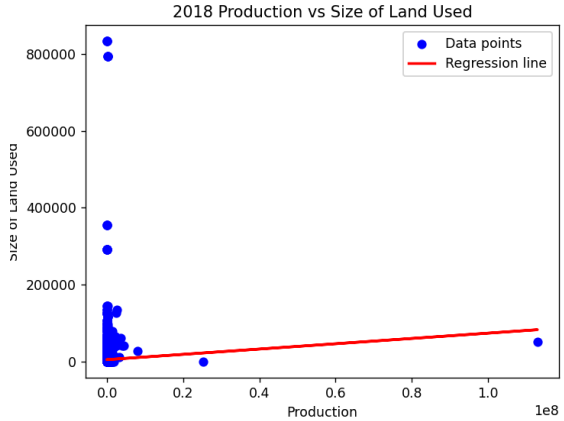

2016 Production vs Price
Data points
Regression line
Price
Production
1e14

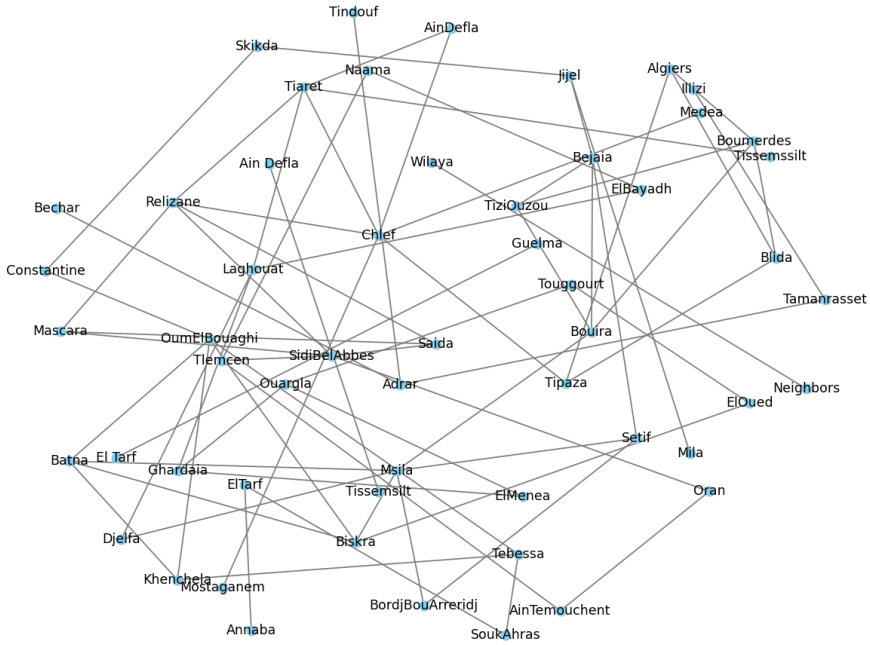

2017 Production vs Consumption
Data points
Regression line
Consumption
Production
1e6

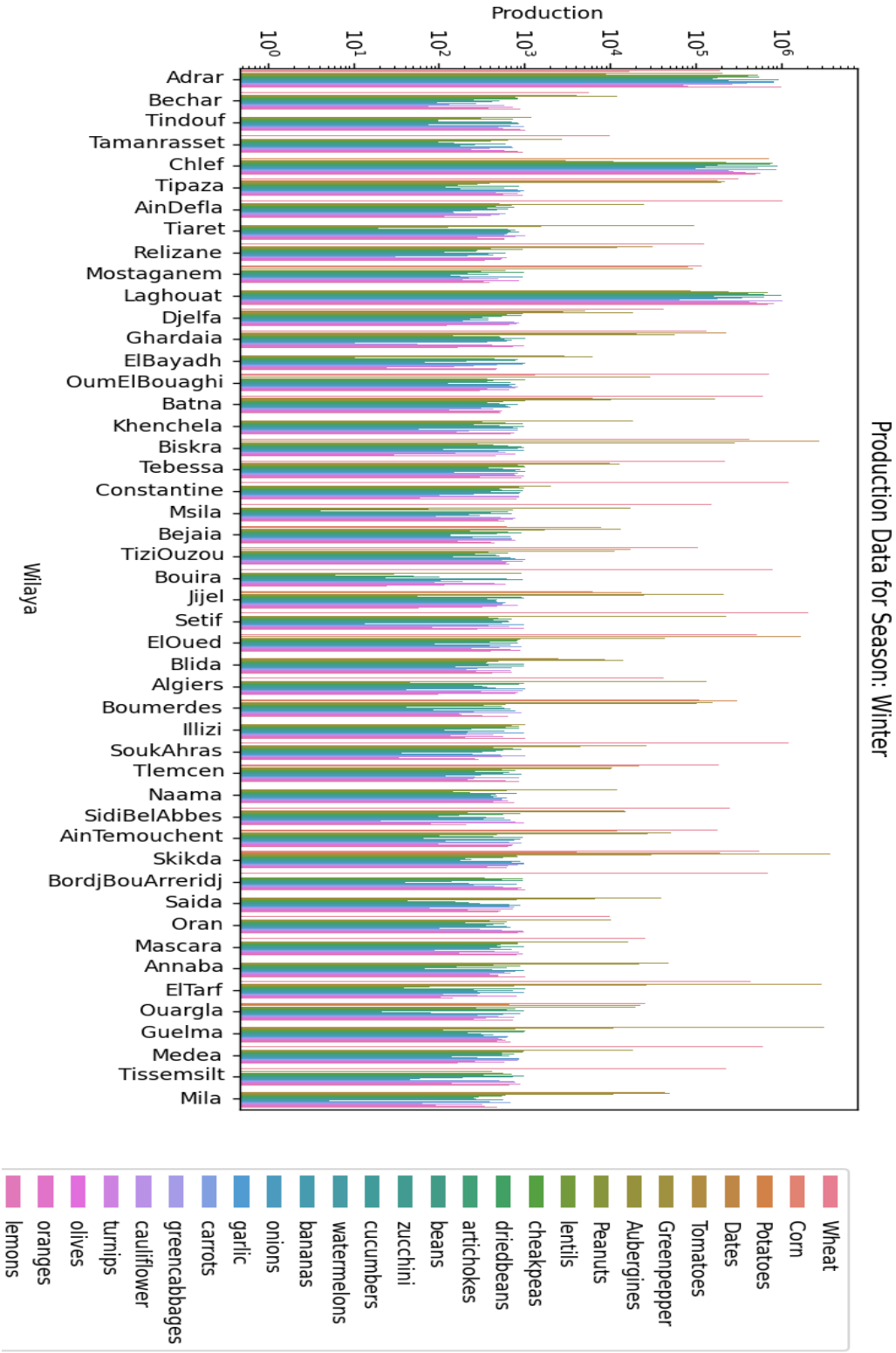

2017 Production vs Size of Land Used
Data points
Regression line
Size of Land Used
Production
1e8


2017 Production vs Price
Data points
Regression line
Price
Production
1e6


2018 Production vs Consumption
Data points
Regression line
Consumption
Production
1e8


2018 Production vs Size of Land Used
Data points
Regression line
Size of Land Used
Production
1e8


Tindouf
AinDefla
Skikda
Naama
Jijel
Algiers
Tiaret
Illizi
Medea
Boumerdes
Tissemssilt
Ain Defla
Wilaya
Bejaia
ElBayadh
Bechar
Relizane
TiziOuzou
Chlef
Guelma
Blida
Constantine
Laghouat
Touggourt
Tamanrasset
Mascara
OumElBouaghi
Saida
Bouira
Tlemcen
SidiBelAbbes
Ouargla
Adrar
Tipaza
Neighbors
ElOued
Setif
Mila
Batna
El Tarf
Ghardaia
Msila
ElTarf
Tissemsilt
ElMenea
Oran
Djelfa
Biskra
Tebessa
Khenchela
Mostaganem
BordjBouArreridj
AinTemouchent
Annaba
SoukAhras


Production Data for Season: Winter
Production
10^0
10^1
10^2
10^3
10^4
10^5
10^6
Wilaya
Adrar
Bechar
Tindouf
Tamanrasset
Chlef
Tipaza
AinDefla
Tiaret
Relizane
Mostaganem
Laghouat
Djelfa
Ghardaia
ElBayadh
OumElBouaghi
Batna
Khenchela
Biskra
Tebessa
Constantine
Msila
Bejaia
TiziOuzou
Bouira
Jijel
Setif
ElOued
Blida
Algiers
Boumerdes
Illizi
SoukAhras
Tlemcen
Naama
SidiBelAbbes
AinTemouchent
Skikda
BordjBouArreridj
Saida
Oran
Mascara
Annaba
ElTarf
Ouargla
Guelma
Medea
Tissemsilt
Mila
Wheat
Corn
Potatoes
Dates
Tomatoes
Greenpepper
Aubergines
Peanuts
lentils
cheakpeas
driedbeans
artichokes
beans
zucchini
cucumbers
watermelons
bananas
onions
garlic
carrots
greencabbages
cauliflower
turnips
olives
oranges
lemons

➢ Conclusion :

Within this AI job concentrated on the company of farming in Algeria, we looked to attend to the difficulty of identifying the optimum farming methods for each and every Wilaya as well as boosting farming performance via innovative formulaic strategies. By leveraging information evaluation coupled with graph-based depictions, we recognized vital farming staminas plus prospective locations for renovation throughout various wilayas.

Our technique included creating an extensive chart that stood for numerous farming features coupled with partnerships in between the wilayas. We used numerous search formulas, consisting of Breadth-First Search (BFS), Depth-First Search (DFS), A *, and also Hill Climbing to get this chart coupled with reveal ideal options for farming companies.

The meta-level tree structure we executed helped with the structuring as well as evaluation of complicated farming information enabling us to effectively check out numerous circumstances as well as end results. With relative evaluation, we showed the staminas and also constraints of each search formula in the context of our particular trouble.

BFS gave a detailed expedition of all feasible options making certain that we did not miss out on any type of possible optimum setups. DFS enabled much deeper expeditions of detailed courses albeit with the threat of neglecting some opportunities as a result of its depth-first nature. The A * formula with its heuristic-driven method, well balanced the thoroughness of BFS with the performance of DFS showing reliability in lots of instances. Hillside Climbing, while helpful for locating regional optima promptly, in some cases fought with determining worldwide optima because of its susceptibility to regional maxima.

Our search highlights the relevance of picking suitable formulas based upon the certain attributes of the farming information as well as the optimization objectives. By integrating these mathematical strategies with a durable meta-level tree framework we had the ability to suggest customized farming approaches for every Wilaya eventually adding to enhanced farming companies along with performance in Algeria.

Finally, this job shows the capacity of AI plus innovative search formulas in taking on intricate, real-world troubles in farming. The understandings obtained from our evaluation supply a useful structure for future research study and also growth, leading the way for even more advanced as well as efficient farming optimization strategies.

Appendix:

References:

https://madr.gov.dz/wp-content/uploads/2022/04/SERIE-B-2016.pdf

https://madr.gov.dz/wp-content/uploads/2022/04/SERIE-B-2017-.pdf

https://madr.gov.dz/wp-content/uploads/2022/04/SERIE-B-2018.pdf

https://madr.gov.dz/wp-content/uploads/2022/04/SERIE-B-2019.pdf

Who did What?

Right from the start, we held meetings every day and conversed about everything collectively. In general, mutual assistance was extended amongst us all:

KHEFFACHE Semhane:

-BFS implementation .

-Generating data2019.csv file.

-coding class “product” and helping in all codes.

-Report's writing and idea figure(1/4).

-Testing & debugging (1/4).

-Visualization (1/4).

HATTABI Hadil:

-DFS implementation .

-Generating data2017.csv file.

-coding the “main” and helping in all codes.

-Report's writing and idea figure(1/4).

-Testing & debugging (1/4).

-Visualization (1/4).

SOUAK Maroua:

-Hill_Climbing implementation .

-Generating data2018.csv file.

-coding class “wilaya” and helping in all codes.

-Report's writing and idea figure(1/4).

-Testing & debugging (1/4).

-Visualization (1/4).

MAHFOUDIA Nour el houda Imene:

-A* implementation .

-Generating data2016.csv file.

-coding the “graph” and meta_level class and helping in all codes.

-Report's writing and idea figure(1/4).

-Testing & debugging (1/4).

-Visualization (1/4).

Screenshots:

```
Welcome to the Production Analysis and Optimization System!
please choose the year you wish to perform a search on (2016 - 2019): 2017
Please choose an option:
1. Best product production in a given wilaya
2. Highest production in a given product
3. Find the best scenario of agriculture in Algeria
4. Does Algeria achieve self-sufficiency in a given product?
5. What is the country's consumption of a given product?:
Enter the number of your choice: 4
Enter product whose self-sufficiency will be checked: Peaches
Algeria was not self-sufficient in Peaches in the given year.
Total production: 66716.0
Total consumption: 89230942.0
Difference (Production - Consumption): -89164226.0
```

```python
def measure_memory_usage():
    mem_usage_before = memory_usage()[0]* 1024 * 1024
    mem_usage_after = memory_usage()[0]* 1024 * 1024
    memory_used = mem_usage_after - mem_usage_before
    return memory_used
```

```python
execution_time = time.time() - start_time
print(f"Execution time: {execution_time:.2f} seconds")
```

```
Welcome to the Production Analysis and Optimization System!
please choose the year you wish to perform a search on (2016 - 2019): 2019
Please choose an option:
1. Best product production in a given wilaya
2. Highest production in a given product
3. Find the best scenario of agriculture in Algeria
4. Does Algeria achieve self-sufficiency in a given product?
5. What is the country's consumption of a given product?:
Enter the number of your choice: 5
Enter product whose total consumption will be summed: Legumes
Total consumption of  Legumes  in the year  2019  :  61664573.0
```

```python
def get_production(self, wilaya, product):
    for p in wilaya.products:
        if p.name == product.name and p.season == product.season:
            return p.production
    return 0

def get_price(self, wilaya, product):
    for p in wilaya.products:
        if p.name == product.name and p.season == product.season:
            return p.price
    return float('inf')  # Return infinity if price not found

def calculate_self_sufficiency(graph, product_name):
    total_production = 0
    total_consumption = 0

    for wilaya in graph.wilayas.values():
        for product in wilaya.products:
            if product.name == product_name:
                total_production += product.production
                total_consumption += product.consumption

    difference = total_production - total_consumption

    if difference > 0:
        print(f"Algeria was self-sufficient in {product_name} in the given year.")
        print(f"Total production: {total_production}")
        print(f"Total consumption: {total_consumption}")
        print(f"Difference (Production - Consumption): {difference}")
    else:
        print(f"Algeria was not self-sufficient in {product_name} in the given year.")
        print(f"Total production: {total_production}")
        print(f"Total consumption: {total_consumption}")
        print(f"Difference (Production - Consumption): {difference}")

def sum_consumption(graph, product_name):
    total_consumption = 0

    for wilaya in graph.wilayas.values():
        for product in wilaya.products:
            if product.name == product_name:
                total_consumption += product.consumption

    return total_consumption
```

```python
def visualize_graph(self):
    G = nx.Graph()
    for wilaya, neighbors in self.adjacency_list.items():
        for neighbor in neighbors:
            G.add_edge(wilaya, neighbor)

    # Increase the distance between nodes
    pos = nx.spring_layout(G, k=12)  # Adjust the value of k as needed

    plt.figure(figsize=(10, 8))
    nx.draw(G, pos, with_labels=True, node_color='skyblue', node_size=50, edge_color='gray', font_size=10)
    plt.title('Wilaya Connections Graph')
    plt.show()


def visualize_production(self, season):
    wilaya_names = []
    product_names = []
    productions = []

    for wilaya in self.wilayas.values():
        for product in wilaya.products:
            if product.season == season:
                wilaya_names.append(wilaya.name)
                product_names.append(product.name)
                productions.append(product.production)

    data = {
        'Wilaya': wilaya_names,
        'Product': product_names,
        'Production': productions
    }

    plt.figure(figsize=(12, 6))
    ax = sns.barplot(x='Wilaya', y='Production', hue='Product', data=data)

    # Rotate x-axis labels vertically for better readability
    ax.set_xticklabels(ax.get_xticklabels(), rotation=90, ha='center')

    # Move the legend outside the plot
    plt.legend(bbox_to_anchor=(1.05, 1), loc='upper left')

    plt.title(f'Production Data for Season: {season}')
    plt.tight_layout()  # Adjust layout to prevent clipping of labels
    plt.yscale('log')
    plt.show()
```

```
Welcome to the Production Analysis and Optimization System!
please choose the year you wish to perform a search on (2016 - 2019): 2016
Please choose an option:
1. Best product production in a given wilaya
2. Highest production in a given product
3. Find the best scenario of agriculture in Algeria
4. Does Algeria achieve self-sufficiency in a given product?
5. What is the country's consumption of a given product?:
Enter the number of your choice: 2
Enter product that will have its wilayas searched: Wheat
Enter season of search: Winter
1_ Breadth-First Search (BFS).
2_ Deapth-First Search (DFS).
3_ Hill Climbing Search.
4_ A* Search.
5_ All at once.
Enter search to be performed: 5
Enter search parameter to be performed:
Enter 1 to find the best wilaya production wise.
Enter 2 to find the best wilaya price wise.
Enter choice: 1
------------------------
Result of DFS Search:
The wilaya with the highest production of Wheat is Setif with a production of 2013972.0
------------------------
BFS Search Result:
The wilaya with the highest production of Wheat is Setif with a production of 2013972.0
------------------------
Hill Climbing Search Result:
The wilaya with the highest production of Wheat is Mila with a production of 1548510.0
------------------------
A* Search Result:
The wilaya with the highest production of Wheat is Setif with a production of 2013972.0
------------------------
Execution time: 18.08 seconds
```

```
A* Search Result:
Adrar is best at producing:
- Wheat with production 179048464698794.0
- Corn with production 1613646.0
- Potatoes with production 9999040.0
- Dates with production 7896522.0
- Greenpepper with production 2093.0
- Aubergines with production 321.0
- Peanuts with production 763207.0
- lentils with production 242895.0
- cheakpeas with production 127920.0
- driedbeans with production 24715.0
- artichokes with production 799062.0
- beans with production 344350.0
- zucchini with production 882633.0
- cucumbers with production 929316.0
- watermelons with production 579092.0
- bananas with production 88995.0
- onions with production 826326.0
- garlic with production 526845.0
- carrots with production 463331.0
- greencabbages with production 567198.0
- cauliflower with production 761386.0
- turnips with production 766336.0
- olives with production 422054.0
- oranges with production 133063.0
- lemons with production 387753.0
- tangerines with production 429584.0
- Tomatoes with production 6468.0
Execution time: 12.99 seconds
Memory Usage (bytes): 0.0
```

```
Welcome to the Production Analysis and Optimization System!
please choose the year you wish to perform a search on (2016 - 2019): 2018
Please choose an option:
1. Best product production in a given wilaya
2. Highest production in a given product
3. Find the best scenario of agriculture in Algeria
4. Does Algeria achieve self-sufficiency in a given product?
5. What is the country's consumption of a given product?:
Enter the number of your choice: 1
Enter wilaya whose products will be searched: Algiers
Enter season of search: Winter
1_ Breadth-First Search (BFS).
2_ Deapth-First Search (DFS).
3_ Hill Climbing Search.
4_ A* Search.
5_ All at once.
Enter search to be performed: 4
Enter search parameter to be performed:
Enter 1 to find the best wilaya production wise.
Enter 2 to find the best wilaya price wise.
Enter choice: 1
A* Search Result:
Algiers is best at producing:
- Dates with production 0.0
- olives with production 926.0
Execution time: 13.19 seconds
```

```python
def a_star_search_wilaya(self, product_name, search_season, search_by):
    def heuristic(wilaya, product, search_by):
        if search_by == 1:
            return -product.production
        elif search_by == 2:
            return product.price

    priority_queue = []
    for wilaya_name in self.wilayas:
        heapq.heappush(priority_queue, (0, wilaya_name))

    visited = set()
    best_production = {}
    best_price = {}

    while priority_queue:
        _, current_wilaya_name = heapq.heappop(priority_queue)
        if current_wilaya_name in visited:
            continue
        visited.add(current_wilaya_name)
        current_wilaya = self.wilayas[current_wilaya_name]

        for product in current_wilaya.products:
            if product.name == product_name and product.season == search_season:
                if search_by == 1:
                    if product_name not in best_production or product.production > best_production[product_name][1]:
                        best_production[product_name] = (current_wilaya.name, product.production)

                elif search_by == 2:
                    if product_name not in best_price or product.price < best_price[product_name][1]:
                        best_price[product_name] = (current_wilaya.name, product.price)

        for neighbor_name in self.adjacency_list[current_wilaya_name]:
            heapq.heappush(priority_queue, (heuristic(self.wilayas[neighbor_name], product, search_by), neighbor_name))

    if best_production:
        for product, (wilaya, production) in best_production.items():
            print(f"The wilaya with the highest production of {product} is {wilaya} with a production of {production}")

    if best_price:
        for product, (wilaya, price) in best_price.items():
            print(f"The wilaya with the lowest price of {product} is {wilaya} with a price of {price}")

    if not best_production and not best_price:
        print(f"No wilaya has the highest production or the lowest price for the product {product_name} in season {search_season}.")



def get_production(self, wilaya, product):
    for p in wilaya.products:
        if p.name == product.name and p.season == product.season:
            return p.production
    return 0

def get_price(self, wilaya, product):
    for p in wilaya.products:
        if p.name == product.name and p.season == product.season:
            return p.price
    return float('inf')  # Return infinity if price not found

def calculate_self_sufficiency(graph, product_name):
    total_production = 0
    total_consumption = 0

    for wilaya in graph.wilayas.values():
        for product in wilaya.products:
            if product.name == product_name:
                total_production += product.production
                total_consumption += product.consumption

    difference = total_production - total_consumption

    if difference > 0:
        print(f"Algeria was self-sufficient in {product_name} in the given year.")
        print(f"Total production: {total_production}")
        print(f"Total consumption: {total_consumption}")
        print(f"Difference (Production - Consumption): {difference}")
    else:
        print(f"Algeria was not self-sufficient in {product_name} in the given year.")
        print(f"Total production: {total_production}")
        print(f"Total consumption: {total_consumption}")
        print(f"Difference (Production - Consumption): {difference}")

def sum_consumption(graph, product_name):
    total_consumption = 0

    for wilaya in graph.wilayas.values():
        for product in wilaya.products:
            if product.name == product_name:
                total_consumption += product.consumption

    return total_consumption
```

```
Enter product name to search: Wheat
THE BEST PRODUCTION SCENARIO OF  Wheat  :
Wilaya: Adrar, Year: 2016, Highest Production: 179048464698794.0, Land Used: 554.0
Wilaya: Chlef, Year: 2018, Highest Production: 1438090.0, Land Used: 65888.0
Wilaya: Laghouat, Year: 2017, Highest Production: 167055.0, Land Used: 23580.0
Wilaya: Oumelbouaghi, Year: 2017, Highest Production: 1900000.0, Land Used: 105000.0
Wilaya: Batna, Year: 2017, Highest Production: 1360170.0, Land Used: 82716.0
Wilaya: Bejaia, Year: 2018, Highest Production: 130867.0, Land Used: 5483.0
Wilaya: Biskra, Year: 2018, Highest Production: 502310.0, Land Used: 12777.0
Wilaya: Bechar, Year: 2017, Highest Production: 40152.0, Land Used: 1440.0
Wilaya: Blida, Year: 2018, Highest Production: 174108.0, Land Used: 6485.0
Wilaya: Bouira, Year: 2018, Highest Production: 1422807.0, Land Used: 43925.0
Wilaya: Tamanrasset, Year: 2018, Highest Production: 11343.0, Land Used: 515.0
Wilaya: Tebessa, Year: 2017, Highest Production: 877650.0, Land Used: 70000.0
Wilaya: Tlemcen, Year: 2017, Highest Production: 916150.0, Land Used: 87000.0
Wilaya: Tiaret, Year: 2016, Highest Production: 7898456321550.0, Land Used: 0.0
Wilaya: Tiziouzou, Year: 2018, Highest Production: 157341.0, Land Used: 7240.0
Wilaya: Algiers, Year: 2018, Highest Production: 48460.0, Land Used: 1691.0
Wilaya: Djelfa, Year: 2017, Highest Production: 551000.0, Land Used: 90810.0
Wilaya: Jijel, Year: 2018, Highest Production: 18160.0, Land Used: 765.0
Wilaya: Setif, Year: 2018, Highest Production: 2458253.0, Land Used: 127416.0
Wilaya: Saida, Year: 2018, Highest Production: 658410.0, Land Used: 32338.0
Wilaya: Skikda, Year: 2018, Highest Production: 583341.0, Land Used: 26550.0
Wilaya: Sidibelabbes, Year: 2017, Highest Production: 676268.0, Land Used: 96820.0
Wilaya: Annaba, Year: 2018, Highest Production: 390863.0, Land Used: 12835.0
Wilaya: Guelma, Year: 2018, Highest Production: 2149081.0, Land Used: 66035.0
Wilaya: Constantine, Year: 2018, Highest Production: 1907808.0, Land Used: 54097.0
Wilaya: Medea, Year: 2018, Highest Production: 1401337.0, Land Used: 69394.0
Wilaya: Mostaganem, Year: 2017, Highest Production: 334050.0, Land Used: 26079.0
Wilaya: Msila, Year: 2017, Highest Production: 457625.0, Land Used: 54254.0
Wilaya: Mascara, Year: 2018, Highest Production: 637000.0, Land Used: 42500.0
Wilaya: Ouargla, Year: 2018, Highest Production: 72023.0, Land Used: 2002.0
Wilaya: Oran, Year: 2018, Highest Production: 217316.0, Land Used: 9957.0
Wilaya: Elbayadh, Year: 2017, Highest Production: 140500.0, Land Used: 12399.0
Wilaya: Illizi, Year: 2017, Highest Production: 486.0, Land Used: 18.0
Wilaya: Bordjbouarreridj, Year: 2018, Highest Production: 787730.0, Land Used: 59097.0
Wilaya: Boumerdes, Year: 2016, Highest Production: 107738.0, Land Used: 4552.0
Wilaya: Eltarf, Year: 2018, Highest Production: 488681.0, Land Used: 22338.0
Wilaya: Tindouf, Year: 2016, Highest Production: 0.0, Land Used: 0.0
Wilaya: Tissemsilt, Year: 2018, Highest Production: 1046096.0, Land Used: 56640.0
Wilaya: Eloued, Year: 2016, Highest Production: 500000.0, Land Used: 10000.0
Wilaya: Khenchela, Year: 2017, Highest Production: 710000.0, Land Used: 45000.0
Wilaya: Soukahras, Year: 2016, Highest Production: 1207706.0, Land Used: 76600.0
Wilaya: Tipaza, Year: 2018, Highest Production: 346873.0, Land Used: 13915.0
Wilaya: Mila, Year: 2018, Highest Production: 1968262.0, Land Used: 65295.0
Wilaya: Aindefla, Year: 2018, Highest Production: 1527458.0, Land Used: 55556.0
Wilaya: Naama, Year: 2017, Highest Production: 35035.0, Land Used: 3615.0
Wilaya: Aintemouchent, Year: 2018, Highest Production: 1172100.0, Land Used: 51400.0
Wilaya: Ghardaia, Year: 2018, Highest Production: 196752.0, Land Used: 4533.0
Wilaya: Relizane, Year: 2018, Highest Production: 1442279.0, Land Used: 78927.0
```

```
BFS Search Result:
Chlef is best at producing:
- Potatoes with production 704760.0
- Tomatoes with production 3709500.0
- cheakpeas with production 764243.0
- driedbeans with production 715871.0
- artichokes with production 667209.0
- zucchini with production 893325.0
- bananas with production 521668.0
- onions with production 656613.0
- carrots with production 844789.0
- oranges with production 957145.0
- Dates with production 2750700.0
- Corn with production 6060.0
- Wheat with production 2013972.0
- Greenpepper with production 894200.0
- Aubergines with production 277340.0
- Peanuts with production 731042.0
- lentils with production 670339.0
- beans with production 969134.0
- cucumbers with production 161734.0
- watermelons with production 615056.0
- garlic with production 178314.0
- greencabbages with production 495660.0
- cauliflower with production 998081.0
- turnips with production 411844.0
- olives with production 556968.0
- lemons with production 799920.0
- tangerines with production 672269.0
Execution time: 16.20 seconds
Memory Usage (bytes): 0.0
```

```
Welcome to the Production Analysis and Optimization System!
please choose the year you wish to perform a search on (2016 - 2019): 2018
Please choose an option:
1. Best product production in a given wilaya
2. Highest production in a given product
3. Find the best scenario of agriculture in Algeria
4. Does Algeria achieve self-sufficiency in a given product?
5. What is the country's consumption of a given product?:
Enter the number of your choice: 1
Enter wilaya whose products will be searched: Adrar
Enter season of search: Spring
1_ Breadth-First Search (BFS).
2_ Deapth-First Search (DFS).
3_ Hill Climbing Search.
4_ A* Search.
5_ All at once.
Enter search to be performed: 2
Enter search parameter to be performed:
Enter 1 to find the best wilaya production wise.
Enter 2 to find the best wilaya price wise.
Enter choice: 2
BFS Search Result:

Adrar is best at providing the lowest price for:
- olives with price 1647.0
- Dates with price 1363.0
Execution time: 21.55 seconds
```

```python
def bfs_search_wilaya(self, product_name, search_season, search_by):
    queue = deque(self.wilayas.values())
    visited = set()
    best_production = {}
    best_price = {}

    while queue:
        current_wilaya = queue.popleft()
        if current_wilaya.name in visited:
            continue
        visited.add(current_wilaya.name)

        for product in current_wilaya.products:
            if product.name == product_name and product.season == search_season:
                if search_by == 1:
                    if product_name not in best_production or product.production > best_production[product_name][1]:
                        best_production[product_name] = (current_wilaya.name, product.production)

                elif search_by == 2:
                    if product_name not in best_price or product.price < best_price[product_name][1]:
                        best_price[product_name] = (current_wilaya.name, product.price)

        for neighbor_name in self.adjacency_list[current_wilaya.name]:
            neighbor_wilaya = self.wilayas[neighbor_name]
            if neighbor_wilaya.name not in visited:
                queue.append(neighbor_wilaya)

    if best_production:
        for product, (wilaya, production) in best_production.items():
            print(f"The wilaya with the highest production of {product} is {wilaya} with a production of {production}")

    if best_price:
        for product, (wilaya, price) in best_price.items():
            print(f"The wilaya with the lowest price of {product} is {wilaya} with a price of {price}")

    if not best_production and not best_price:
        print(f"No wilaya has the highest production or the lowest price for the product {product_name} in season {search_season}.")
```

```python
class Graph:
    def __init__(self):
        self.wilayas = {}
        self.adjacency_list = {}

    def add_wilaya(self, wilaya_name):
        if wilaya_name not in self.wilayas:
            self.wilayas[wilaya_name] = Wilaya(wilaya_name)
            self.adjacency_list[wilaya_name] = []

    def add_product_to_wilaya(self, wilaya_name, product):
        if wilaya_name in self.wilayas:
            self.wilayas[wilaya_name].add_product(product)

    def connect_wilayas(self, wilaya1, wilaya2):
        if wilaya1 in self.wilayas and wilaya2 in self.wilayas:
            self.adjacency_list[wilaya1].append(wilaya2)
            self.adjacency_list[wilaya2].append(wilaya1)
```

```python
class Product:
    def __init__(self, name, production, season, price, consumption):
        self.name = name
        self.production = production
        self.season = season
        self.price = price
        self.consumption = consumption
```

```python
class Wilaya:
    def __init__(self, name):
        self.name = name
        self.products = []

    def add_product(self, product):
        self.products.append(product)
```

```python
def load_graph_from_csv(filename):
    graph = Graph()

    with open(filename, mode='r', newline='', encoding='utf-8') as file:
        csv_reader = csv.reader(file, delimiter=',')
        for row in csv_reader:
            if row and row[0]:  # Check if the row is not empty and the first element exists
                wilaya = row[0].strip()  # The first item is the wilaya
                if len(row) > 1:
                    neighbors_raw = row[1].strip().strip('"')  # Strip the outer quotes
                    neighbors = [neighbor.strip() for neighbor in neighbors_raw.split(',')]
                    graph.add_wilaya(wilaya)
                    for neighbor in neighbors:
                        graph.add_wilaya(neighbor)
                        graph.connect_wilayas(wilaya, neighbor)  # Establish connections between wilaya and its neighbors

    return graph

def load_wilaya_data_from_csv(filename, year, graph):
    with open(filename, mode='r', newline='', encoding='utf-8') as file:
        csv_reader = csv.reader(file, delimiter=',')
        headers = next(csv_reader)
        for row in csv_reader:
            if len(row) >= 7:  # Ensure the row has enough elements
                wilaya_name = row[0].strip()
                product_name = row[1].strip()
                try:
                    production = float(row[2]) if row[2] else 0.0
                    price = float(row[5]) if row[5] else 0.0
                    consumption = float(row[6]) if row[6] else 0.0
                except ValueError:
                    # Skip the row if conversion fails
                    print(f"Skipping row due to invalid data: {row}")
                    continue
                season = row[4].strip()
                product = Product(product_name, production, season, price, consumption)
                graph.add_wilaya(wilaya_name)
                graph.add_product_to_wilaya(wilaya_name, product)
```

```
Result of DFS Search:
Chlef is best at producing:
- Potatoes with production 704760.0
- Tomatoes with production 3709500.0
- cheakpeas with production 764243.0
- driedbeans with production 715871.0
- artichokes with production 667209.0
- zucchini with production 893325.0
- bananas with production 521668.0
- onions with production 656613.0
- carrots with production 844789.0
- oranges with production 957145.0
- Dates with production 2750700.0
- Corn with production 6060.0
- Greenpepper with production 894200.0
- Aubergines with production 277340.0
- turnips with production 411844.0
- lemons with production 799920.0
- beans with production 969134.0
- greencabbages with production 495660.0
- watermelons with production 615056.0
- tangerines with production 672269.0
- Peanuts with production 731042.0
- lentils with production 670339.0
- cucumbers with production 161734.0
- garlic with production 178314.0
- cauliflower with production 998081.0
- olives with production 556968.0
- Wheat with production 2013972.0
Execution time: 18.20 seconds
Memory Usage (bytes): 0.0
```

```
Welcome to the Production Analysis and Optimization System!
please choose the year you wish to perform a search on (2016 - 2019): 2016
Please choose an option:
1. Best product production in a given wilaya
2. Highest production in a given product
3. Find the best scenario of agriculture in Algeria
4. Does Algeria achieve self-sufficiency in a given product?
5. What is the country's consumption of a given product?:
Enter the number of your choice: 1
Enter wilaya whose products will be searched: Chlef
Enter season of search: Winter
1_ Breadth-First Search (BFS).
2_ Deapth-First Search (DFS).
3_ Hill Climbing Search.
4_ A* Search.
5_ All at once.
Enter search to be performed: 1
Enter search parameter to be performed:
Enter 1 to find the best wilaya production wise.
Enter 2 to find the best wilaya price wise.
Enter choice: 1
Result of DFS Search:
Chlef is best at producing:
- Potatoes with production 704760.0
- Tomatoes with production 3709500.0
- cheakpeas with production 764243.0
- driedbeans with production 715871.0
- artichokes with production 667209.0
- zucchini with production 893325.0
- bananas with production 521668.0
- onions with production 656613.0
- carrots with production 844789.0
- oranges with production 957145.0
- Dates with production 2750700.0
- Corn with production 6060.0
- Greenpepper with production 894200.0
- Aubergines with production 277340.0
- turnips with production 411844.0
- lemons with production 799920.0
- beans with production 969134.0
- greencabbages with production 495660.0
- watermelons with production 615056.0
- tangerines with production 672269.0
- Peanuts with production 731042.0
- lentils with production 670339.0
- cucumbers with production 161734.0
- garlic with production 178314.0
- cauliflower with production 998081.0
- olives with production 556968.0
- Wheat with production 2013972.0
Execution time: 41.25 seconds
```

```python
def dfs_search_wilaya(self, product_name, search_season, search_by):
    stack = list(self.wilayas.values())
    visited = set()
    best_production = {}
    best_price = {}

    while stack:
        current_wilaya = stack.pop()
        if current_wilaya.name in visited:
            continue
        visited.add(current_wilaya.name)

        for product in current_wilaya.products:
            if product.name == product_name and product.season == search_season:
                if search_by == 1:
                    if product_name not in best_production or product.production > best_production[product_name][1]:
                        best_production[product_name] = (current_wilaya.name, product.production)

                elif search_by == 2:
                    if product_name not in best_price or product.price < best_price[product_name][1]:
                        best_price[product_name] = (current_wilaya.name, product.price)

        for neighbor_name in self.adjacency_list[current_wilaya.name]:
            neighbor_wilaya = self.wilayas[neighbor_name]
            if neighbor_wilaya.name not in visited:
                stack.append(neighbor_wilaya)

    if best_production:
        for product, (wilaya, production) in best_production.items():
            print(f"The wilaya with the highest production of {product} is {wilaya} with a production of {production}")

    if best_price:
        for product, (wilaya, price) in best_price.items():
            print(f"The wilaya with the lowest price of {product} is {wilaya} with a price of {price}")

    if not best_production and not best_price:
        print(f"No wilaya has the highest production or the lowest price for the product {product_name} in season {search_season}.")
```

```
Hill Climbing Search Result:
Adrar is best at producing:
- Wheat with production 179048464698794.0
Execution time: 14.75 seconds
Memory Usage (bytes): 0.0
```

```
Welcome to the Production Analysis and Optimization System!
please choose the year you wish to perform a search on (2016 - 2019): 2017
Please choose an option:
1. Best product production in a given wilaya
2. Highest production in a given product
3. Find the best scenario of agriculture in Algeria
4. Does Algeria achieve self-sufficiency in a given product?
5. What is the country's consumption of a given product?:
Enter the number of your choice: 1
Enter wilaya whose products will be searched: Algiers
Enter season of search: Summer
1_ Breadth-First Search (BFS).
2_ Deapth-First Search (DFS).
3_ Hill Climbing Search.
4_ A* Search.
5_ All at once.
Enter search to be performed: 3
Enter search parameter to be performed:
Enter 1 to find the best wilaya production wise.
Enter 2 to find the best wilaya price wise.
Enter choice: 1
Hill Climbing Search Result:
Algiers does not have the highest production for any product.
Algiers does not have the highest production or the lowest price for any product.
Execution time: 18.04 seconds
```

```python
def hill_climbing_search_wilaya(self, product_name, search_season, search_by):
    best_production = {}
    best_price = {}

    for wilaya in self.wilayas.values():
        if search_by == 1:
            max_production = 0
            best_product = None

            for product in wilaya.products:
                if product.name == product_name and product.season == search_season:
                    product_production = product.production
                    if product_production > max_production:
                        max_production = product_production
                        best_product = product

            if best_product is not None:
                best_production[best_product.name] = (wilaya.name, max_production)

        elif search_by == 2:
            min_price = float('inf')
            best_product = None

            for product in wilaya.products:
                if product.name == product_name and product.season == search_season:
                    product_price = product.price
                    if product_price < min_price:
                        min_price = product_price
                        best_product = product

            if best_product is not None:
                best_price[best_product.name] = (wilaya.name, min_price)

    if best_production:
        for product, (wilaya, production) in best_production.items():
            print(f"The wilaya with the highest production of {product} is {wilaya} with a production of {production}")

    if best_price:
        for product, (wilaya, price) in best_price.items():
            print(f"The wilaya with the lowest price of {product} is {wilaya} with a price of {price}")

    if not best_production and not best_price:
        print(f"No wilaya has the highest production or the lowest price for the product {product_name} in season {search_season}.")
```

```python
# Prompt user for the year and load the corresponding data
print("Welcome to the Production Analysis and Optimization System!")
year = int(input("please choose the year you wish to perform a search on (2016 - 2019): "))
if year in [2016, 2017, 2018, 2019]:
    load_wilaya_data_from_csv(f'{year}.csv', year, graph)
else:
    print("Enter a valid year between 2016 and 2019!")
    return

print("Please choose an option:")
print("1. Best product production in a given wilaya")
print("2. Highest production in a given product")
print("3. Find the best scenario of agriculture in Algeria")
print("4. Does Algeria achieve self-sufficiency in a given product?")
print("5. What is the country's consumption of a given product?:")
choice = input("Enter the number of your choice: ")
```

```python
class Tree:
    def __init__(self, root=None):
        self.root = root

    def add_node(self, parent_name, graph, name, year):
        parent_node = self.find_node(self.root, parent_name)
        if parent_node is not None:
            new_node = TreeNode(graph, name, year)
            parent_node.add_child(new_node)
        else:
            print(f"Parent node '{parent_name}' not found.")

    def find_node(self, node, name):
        if node is None:
            return None
        if node.name == name:
            return node
        for child in node.children:
            result = self.find_node(child, name)
            if result is not None:
                return result
        return None

    def traverse(self, node, level=0):
        if node is not None:
            print(" " * level * 2 + node.name)
            for child in node.children:
                self.traverse(child, level + 1)
```

```python
def search_product(self, product_name):
    highest_production_by_wilaya = {}

    def search_node(node):
        for wilaya in node.graph.wilayas.values():
            for product in wilaya.products:
                if product.name == product_name:
                    if wilaya.name not in highest_production_by_wilaya:
                        highest_production_by_wilaya[wilaya.name] = {
                            "highest_production": product.production,
                            "year": node.year,
                            "land_used": product.size_of_land_used
                        }
                    else:
                        if product.production > highest_production_by_wilaya[wilaya.name]["highest_production"]:
                            highest_production_by_wilaya[wilaya.name] = {
                                "highest_production": product.production,
                                "year": node.year,
                                "land_used": product.size_of_land_used
                            }
        for child in node.children:
            search_node(child)

    search_node(self.root)

    return highest_production_by_wilaya
```